



CS2009

DESIGN AND ANALYSIS OF ALGORITHMS

Dr. Hashim Yasin

**National University of Computer
and Emerging Sciences,
Faisalabad, Pakistan.**

DIVIDE AND
CONQUER



Divide and Conquer

3

- You divide your task (by getting them to distrust each other) and then conquer them piece by piece.
- This is called divide-and-conquer.
- In algorithm design, the idea is to take a problem on a large input,
 - ▣ break the input into smaller pieces,
 - ▣ solve the problem on each of the small pieces,
 - ▣ and then combine the piecewise solutions into a global solution

MERGE SORT



Merge Sort

5

- **Divide:** the problem into a small number of pieces
- **Conquer:** solve each piece by applying divide and conquer to it recursively
- **Combine:** the pieces together into a global solution

Merge Sort

- **Divide:** split A down the middle into two subsequences, each of size roughly $n/2$
- **Conquer:** sort each subsequence by calling merge sort recursively on each.
- **Combine:** merge the two sorted subsequences into a single sorted list

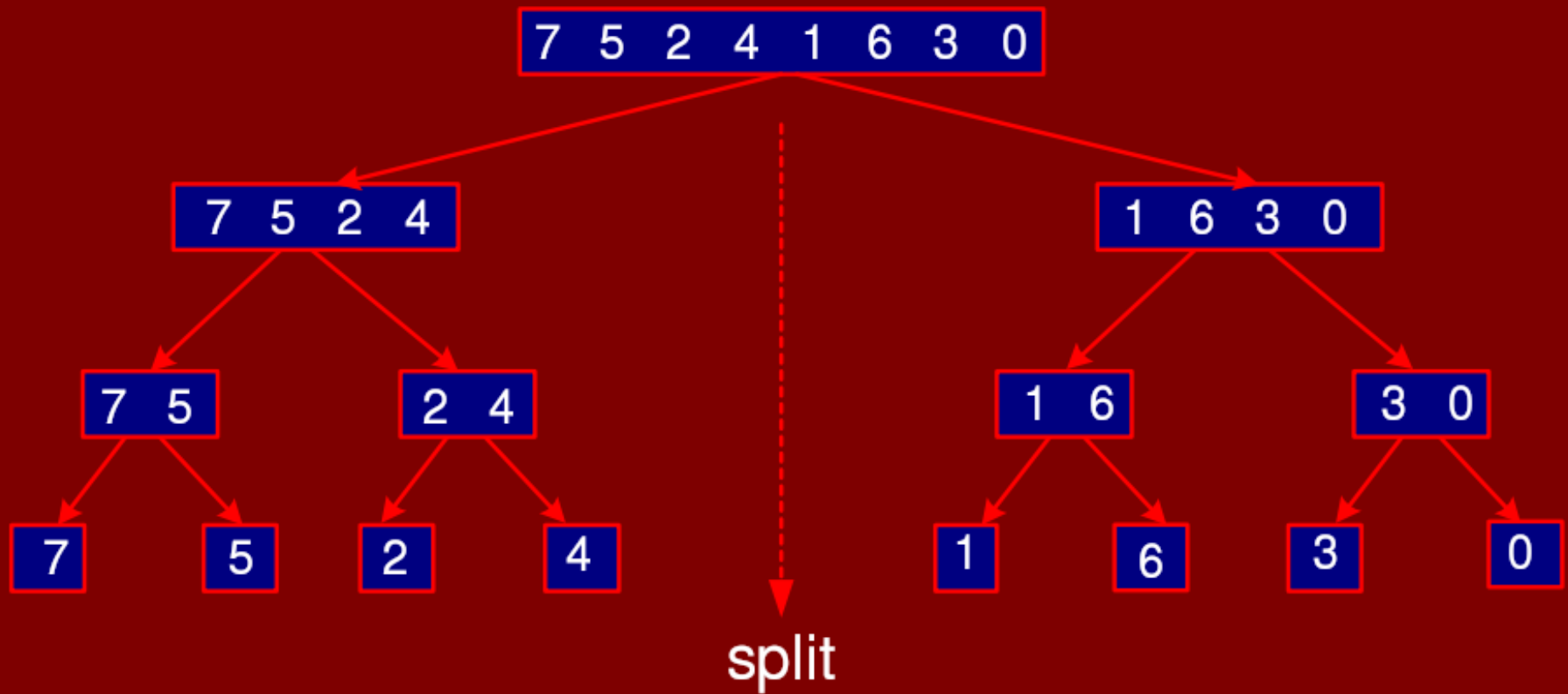
Merge Sort

6

6 5 3 1 8 7 2 4

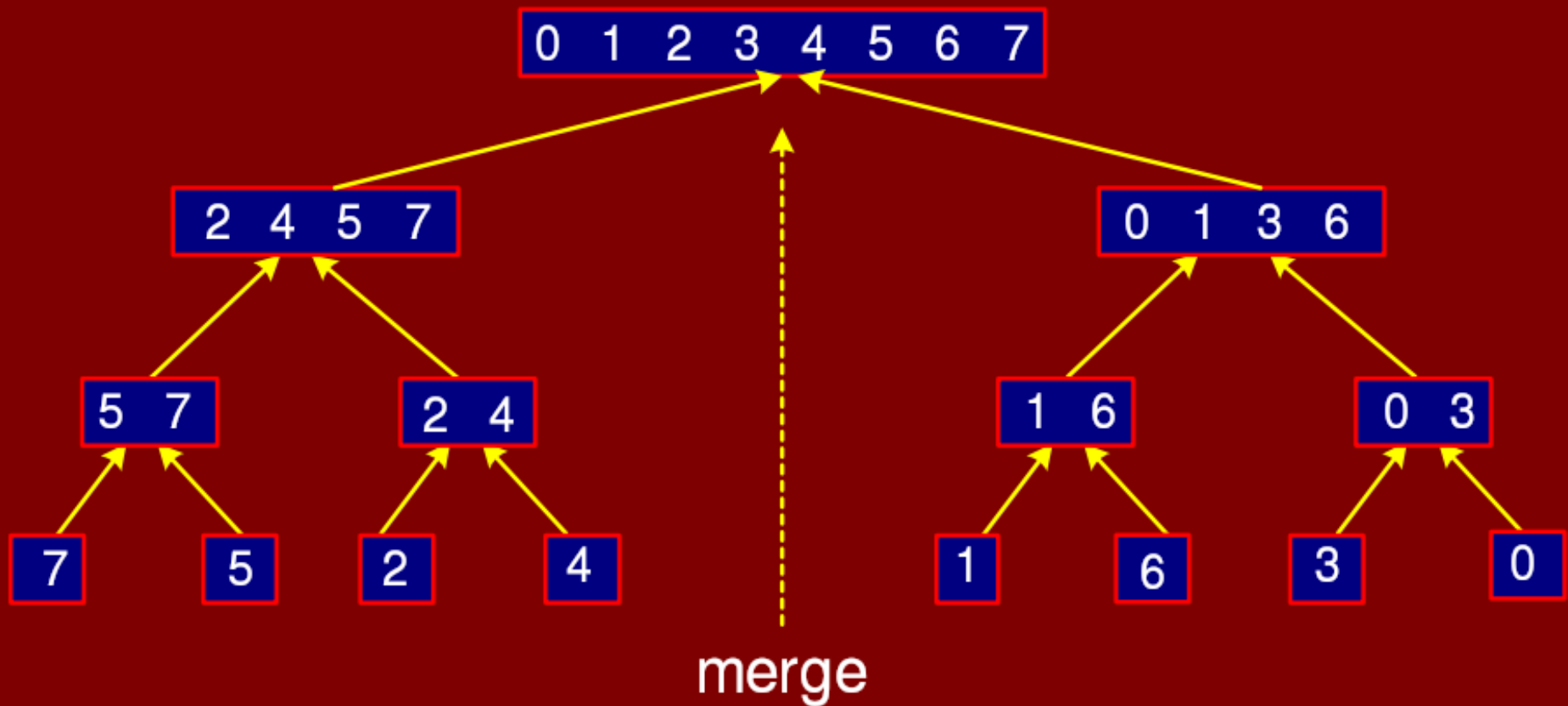
Merge Sort ... Split

7



Merge Sort ... Merge

8



Merge Sort

9

- The fundamental operation in this algorithm is **merging two sorted lists**.
- Because the lists are sorted, this can be done in one pass through the input, if the output is put in a third list.
- An execution of merge-sort is depicted by a binary tree
 - each node represents a recursive call of merge-sort and stores
 - unsorted sequence before the execution and its partition
 - sorted sequence at the end of the execution
 - the root is the initial call
 - the leaves are calls on subsequences of size 0 or 1

Merge Sort

10

- The basic merging algorithm takes
 - ▣ two input arrays: a and b ,
 - ▣ an output array: c
 - ▣ three counters: $aptr$, $bptr$, and $cptr$,
 - which are initially set to the beginning of their respective arrays.
- The smaller of $a[aptr]$ and $b[bptr]$ is copied to the next entry in c , and the appropriate counters are advanced.
- When either input list is exhausted, the remainder of the other list is copied to c .

Merge Sort

11

```
void m_sort( input_type a[], input_type tmp_array[ ], int left, int right )
{
    int center;
    if( left < right )
    {
        center = (left + right) / 2;
        m_sort( a, tmp_array, left, center );
        m_sort( a, tmp_array, center+1, right );
        merge( a, tmp_array, left, center+1, right );
    }
}
```

Merge Sort Example (recursive Function Calls)

Mergesort(a, 8)

m_sort(a, tmp_array, 0, 7)

62 (0)	58 (1)	55 (2)	10 (3)	45 (4)	44 (5)	6 (6)	90 (7)
--------	--------	--------	--------	--------	--------	-------	--------

Merge(a, tmp_array, 0, 4, 7)

m_sort(a, tmp_array, 0, 3)

m_sort(a, tmp_array, 4, 7)

62 (0)	58 (1)	55 (2)	10 (3)	45 (4)	44 (5)	6 (6)	90 (7)
--------	--------	--------	--------	--------	--------	-------	--------

62 (0)	58 (1)	55 (2)	10 (3)	45 (4)	44 (5)	6 (6)	90 (7)
--------	--------	--------	--------	--------	--------	-------	--------

m_sort(0, 1)

m_sort(2, 3)

m_sort(4, 5)

m_sort(6, 7)

Merge(a, tmp_array, 0, 2, 3)

Merge(a, tmp_array, 4, 6, 7)

62 (0)	58 (1)	55 (2)	10 (3)	45 (4)	44 (5)	6 (6)	90 (7)
--------	--------	--------	--------	--------	--------	-------	--------

m_sort(0,0)

m_sort(1,1)

m_sort(2,2)

m_sort(3,3)

m_sort(4,4)

m_sort(5,5)

m_sort(6,6)

m_sort(7,7)

Merge(0, 1, 1)

Merge(2, 3, 3)

Merge(4, 5, 5)

Merge(6, 7, 7)

Merge Sort Example (Merging process)

Mergesort(a, 8)

m_sort(a, tmp_array, 0, 7)

6 (0)	10 (1)	44 (2)	45 (3)	55 (4)	58 (5)	62 (6)	90 (7)
-------	--------	--------	--------	--------	--------	--------	--------

Merge(a, tmp_array, 0, 4, 7)

m_sort(a, tmp_array, 0, 3)

m_sort(a, tmp_array, 4, 7)

62 (0)	58 (1)	55 (2)	10 (3)
--------	--------	--------	--------

45 (4)	44 (5)	6 (6)	90 (7)
--------	--------	-------	--------

62 (0)	58 (1)
--------	--------

55 (2)	10 (3)
--------	--------

45 (4)

44 (5)

6 (6)

90 (7)

m_sort(0, 1)

m_sort(2, 3)

m_sort(4, 5)

m_sort(6, 7)

10 (0)	55 (1)	58 (2)	62 (3)
--------	--------	--------	--------

6 (4)	44 (5)	45 (6)	90 (7)
-------	--------	--------	--------

Merge(a, tmp_array, 0, 2, 3)

Merge(a, tmp_array, 4, 6, 7)

62 (0)	58 (1)	55 (2)	10 (3)
--------	--------	--------	--------

45 (4)

44 (5)

6 (6)

90 (7)

m_sort(0,0)

m_sort(1,1)

m_sort(2,2)

m_sort(3,3)

m_sort(4,4)

m_sort(5,5)

m_sort(6,6)

m_sort(7,7)

Merge(0, 1, 1)

Merge(2, 3, 3)

Merge(4, 5, 5)

Merge(6, 7, 7)

58 (0)	62 (1)
--------	--------

10 (2)	55 (3)
--------	--------

44 (4)	45 (5)
--------	--------

6 (6)

90 (7)

Merge Sort

14

- The conquer step of merge-sort consists of merging two sorted sequences A and B into a sorted sequence S containing the union of the elements of A and B
- Merging two sorted sequences, each with $n/2$ elements and implemented by means of a doubly linked list, takes $O(n)$ time

Merge Sort

15

Algorithm merge(S_1, S_2, S):

Input: Two arrays, S_1 and S_2 , of size n_1 and n_2 , respectively, sorted in non-decreasing order, and an empty array, S , of size at least $n_1 + n_2$

Output: S , containing the elements from S_1 and S_2 in sorted order

$i \leftarrow 1$

$j \leftarrow 1$

while $i \leq n$ **and** $j \leq n$ **do**

if $S_1[i] \leq S_2[j]$ **then**

$S[i + j - 1] \leftarrow S_1[i]$

$i \leftarrow i + 1$

else

$S[i + j - 1] \leftarrow S_2[j]$

$j \leftarrow j + 1$

while $i \leq n$ **do**

$S[i + j - 1] \leftarrow S_1[i]$

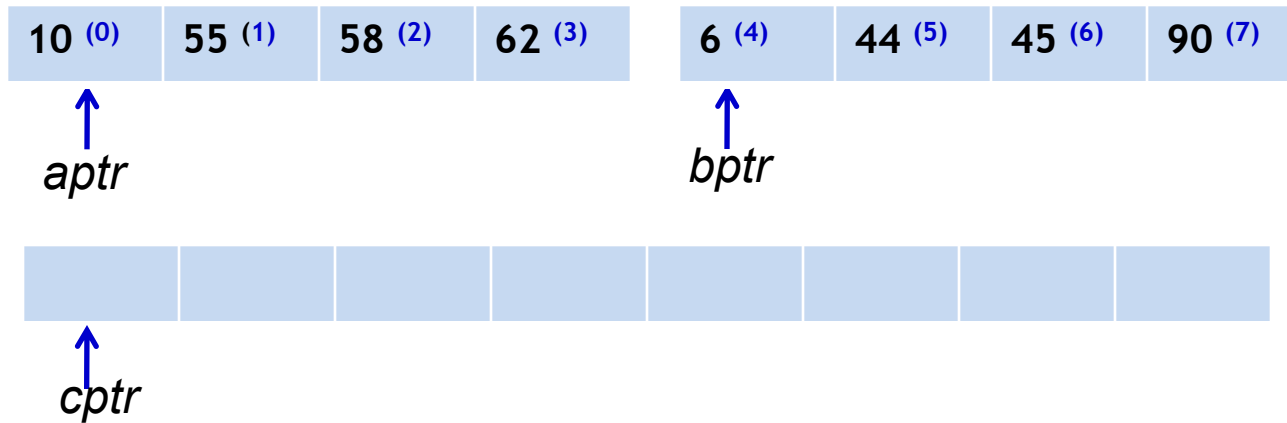
$i \leftarrow i + 1$

while $j \leq n$ **do**

$S[i + j - 1] \leftarrow S_2[j]$

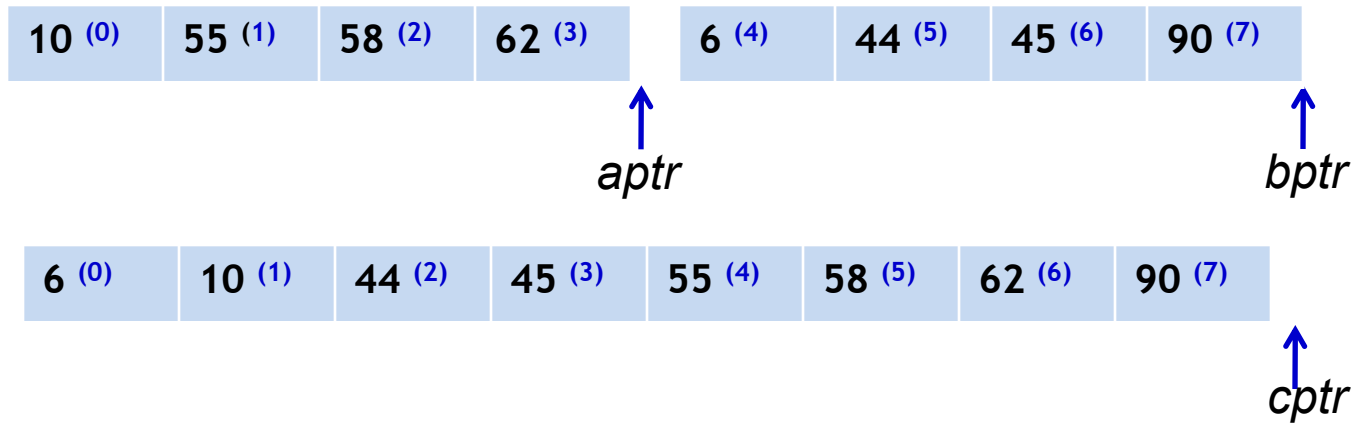
$j \leftarrow j + 1$

Merge two arrays



62 (0)	58 (1)	55 (2)	10 (3)	45 (4)	44 (5)	6 (6)	90 (7)
--------	--------	--------	--------	--------	--------	-------	--------

Merge two arrays



Merge Sort

18

□ **Divide:**

- compute q as the average of p and r : $D(n) = \Theta(1)$

□ **Conquer:**

- recursively solve 2 subproblems, each of size $n/2 \Rightarrow 2T(n/2)$

□ **Combine:**

- MERGE on an n -element subarray takes

- $\Theta(n)$ time $\Rightarrow C(n) = \Theta(n)$

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

Merge Sort

19

$$T(n) = \begin{cases} c & \text{if } n = 1 \\ 2T(n/2) + cn & \text{if } n > 1 \end{cases}$$

Use Master's Theorem:

Compare n with $f(n) = cn$

Case 2: $T(n) = \Theta(n \lg n)$

Merge Sort

20

$$T(n) = \begin{cases} c & \text{if } n = 1 \\ 2T(n/2) + cn & \text{if } n > 1 \end{cases}$$

Use Master's Theorem:

- $T(n) = 2T\left(\frac{n}{2}\right) + n$
- $a = 2, b = 2$, and $f(n) = \theta(n) = \theta(n \log^0 n)$
- $\log_2 2 = 1, k = 1, p = 0$
- $\log_b a = k, p = 0$

Case 2: If $\log_b a = k$, and

- $p > (-1)$, then $\theta(n^k \log^{p+1} n)$
- $\theta(n \log n)$

Acknowledgement

21

Introduction to Algorithms

Thomas H. Cormen, Charles E. Leiserson,
Ronald L. Rivest, and Clifford Stein